

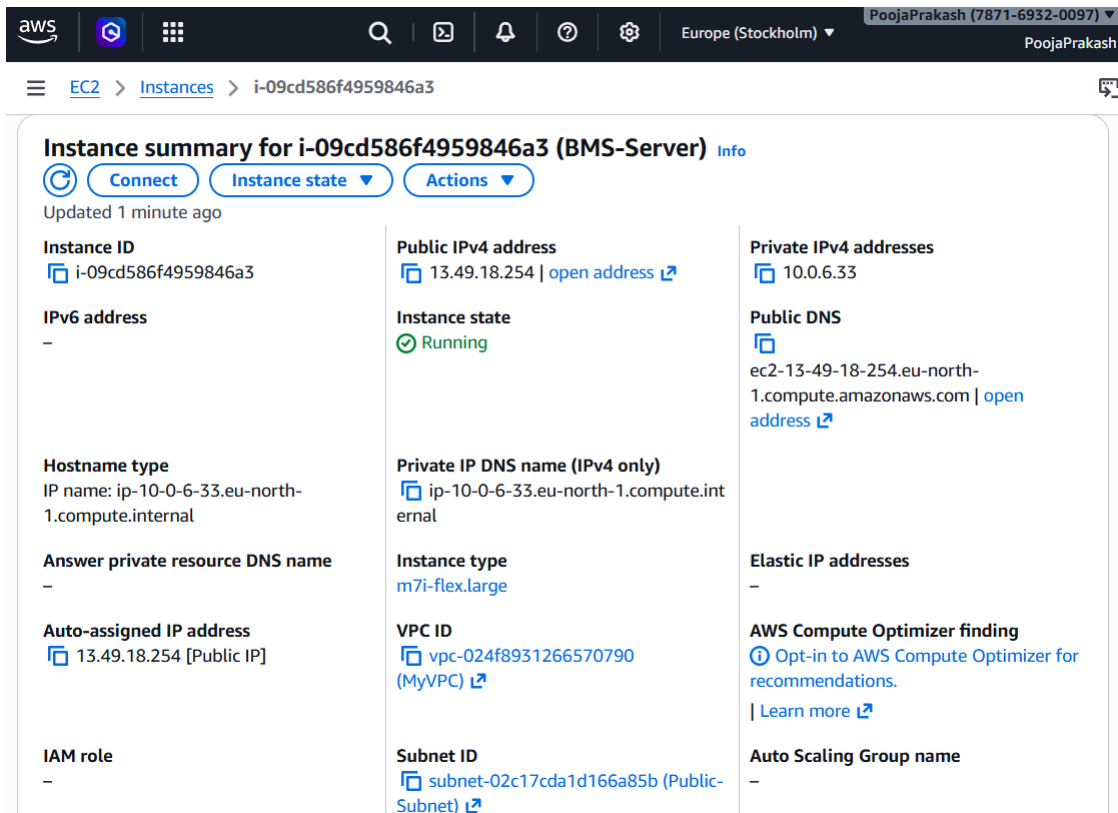
CI/CD Implementation and Kubernetes Deployment of Book My Show Application

Name: Pooja Prakash Belgaumkar

This project demonstrates the end-to-end deployment and monitoring of the Book My Show application using modern DevOps practices. The objective of this project is to implement Continuous Integration, Continuous Deployment, containerization, security scanning, monitoring and Kubernetes orchestration using industry-standard tools such as Jenkins, Docker, SonarQube, OWASP, Trivy, Prometheus, Grafana and Amazon EKS.

Phase 1: CI/CD Setup on BMS Server

- Launched an Ubuntu EC2 instance named BMS-Server and configured necessary inbound rules.



The screenshot displays the AWS Management Console interface for an EC2 instance. The top navigation bar shows the AWS logo, navigation icons, a search bar, and the user's profile (PoojaPrakash (7871-6932-0097)). The breadcrumb trail indicates the path: EC2 > Instances > i-09cd586f4959846a3. The main content area is titled "Instance summary for i-09cd586f4959846a3 (BMS-Server)" and includes buttons for "Connect", "Instance state", and "Actions". Below the title, it states "Updated 1 minute ago". The instance details are organized into three columns:

Instance ID	Public IPv4 address	Private IPv4 addresses
i-09cd586f4959846a3	13.49.18.254 open address	10.0.6.33
IPv6 address	Instance state	Public DNS
-	Running	ec2-13-49-18-254.eu-north-1.compute.amazonaws.com open address
Hostname type	Private IP DNS name (IPv4 only)	Elastic IP addresses
IP name: ip-10-0-6-33.eu-north-1.compute.internal	ip-10-0-6-33.eu-north-1.compute.internal	-
Answer private resource DNS name	Instance type	AWS Compute Optimizer finding
-	m7i-flex.large	Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address	VPC ID	Auto Scaling Group name
13.49.18.254 [Public IP]	vpc-024f8931266570790 (MyVPC)	-
IAM role	Subnet ID	
-	subnet-02c17cda1d166a85b (Public-Subnet)	

Inbound rules (8)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sgr-0d160482a5cd1d125	IPv4	SMTPS	TCP	465	0.0.0.0/0	Email Communication
☐	-	sgr-0b1713ed51333c97d	IPv4	HTTPS	TCP	443	0.0.0.0/0	-
☐	-	sgr-07f169caf5294386b	IPv4	SSH	TCP	22	0.0.0.0/0	-
☐	-	sgr-0f0aaef6a1951089e	IPv4	Custom TCP	TCP	30000 - 32767	0.0.0.0/0	Nodeport Services
☐	-	sgr-043f41d596d4d317e	IPv4	SMTP	TCP	25	0.0.0.0/0	Email Communication
☐	-	sgr-0ad015c03e7a79c3b	IPv4	HTTP	TCP	80	0.0.0.0/0	-
☐	-	sgr-0b3e0d6bb16d32ecf	IPv4	Custom TCP	TCP	6443	0.0.0.0/0	-
☐	-	sgr-08e515c7454082645	IPv4	Custom TCP	TCP	3000 - 10000	0.0.0.0/0	Grafana-Jenkins-SonarQube-Prometheus

- Connected to the instance using MobaXterm and installed Java and Jenkins.

```

root@ip-10-0-6-33:/home/ubuntu# java --version
openjdk 21.0.10 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
root@ip-10-0-6-33:/home/ubuntu# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-02-28 08:39:16 UTC; 2min 8s ago
     Main PID: 5089 (java)
       Tasks: 48 (limit: 9283)
      Memory: 1.2G (peak: 1.2G)
         CPU: 19.429s
    CGroup: /system.slice/jenkins.service
            └─5089 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/

Feb 28 08:41:23 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:23.221+0000 [id=87] INFO h.model
Feb 28 08:41:23 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:23.221+0000 [id=87] INFO h.model
Feb 28 08:41:23 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:23.406+0000 [id=87] INFO h.model
Feb 28 08:41:23 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:23.406+0000 [id=87] INFO h.m.Upd
Feb 28 08:41:23 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:23.406+0000 [id=87] INFO h.m.Upd
Feb 28 08:41:24 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:24.318+0000 [id=87] INFO h.model
Feb 28 08:41:24 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:24.318+0000 [id=87] INFO h.model
Feb 28 08:41:24 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:24.322+0000 [id=87] INFO h.model
Feb 28 08:41:24 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:24.322+0000 [id=87] INFO h.model
Feb 28 08:41:24 ip-10-0-6-33 jenkins[5089]: 2026-02-28 08:41:24.323+0000 [id=87] INFO h.m.Upd
lines 1-20/20 (END)

```

- Installed Docker and logged in to Docker Hub using valid credentials to push Docker images.

```
root@ip-10-0-6-33:/home/ubuntu# docker --version
Docker version 29.2.1, build a5c7197
root@ip-10-0-6-33:/home/ubuntu# docker login -u poojaprakash08

Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-10-0-6-33:/home/ubuntu#
```

- Installed Trivy for file system vulnerability scanning and SonarQube for code quality analysis.

```
root@ip-10-0-6-33:/home/ubuntu# trivy --version
Version: 0.69.1
root@ip-10-0-6-33:/home/ubuntu# docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
Unable to find image 'sonarqube:lts-community' locally
lts-community: Pulling from library/sonarqube
f3929ce9ef98: Pull complete
e24a8b9e652f: Pull complete
4f4fb700ef54: Pull complete
1df735f481ad: Pull complete
5d5a1fad7028: Pull complete
eb27e3a98da1: Pull complete
c7ad1fe61e07: Pull complete
60d98d907669: Pull complete
600f318704dc: Download complete
ce2bd68be4fc: Download complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
315ece51e208029f3f24409e7e8768621bb48748dbf8805185daa1d2dc2eeeb9
root@ip-10-0-6-33:/home/ubuntu# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
sonarqube:lts-community	f709975ab31d	1.02GB	398MB	U

```
root@ip-10-0-6-33:/home/ubuntu# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
315ece51e208	sonarqube:lts-community	"/opt/sonarqube/dock..."	20 seconds ago	Up 15 seconds	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp

```
root@ip-10-0-6-33:/home/ubuntu#
```

- Accessed:
 - ✓ Jenkins via port 8080
 - ✓ SonarQube via port 9000

- Generated a SonarQube authentication token and added it to Jenkins credentials.
- Also added Docker Hub credentials in Jenkins.

Tokens of Administrator

Generate Tokens

Name Expires in

30 days Generate

! New token "token" has been created. Make sure you copy it now, you won't be able to see it again!

Copy squ_1254facf95ac9a7096d945198ce56097b99a1906

Name	Type	Project	Last use	Created	Expiration	
token	User		Never	February 28, 2026	March 30, 2026	Revoke

[Done](#)

Jenkins / [Manage Jenkins](#) / [Credentials](#) / [System](#) / [Global](#)

🔍 ⚙️ 👤

Global

Credentials that should be available everywhere.

+ Add Credentials

📄 Sonat-token
Sonat-token

⚙️ ⋮

📄 docker pooojaprakash08/*****
docker

⚙️ ⋮

- Created a webhook in SonarQube and configured it with the Jenkins URL to trigger pipeline updates automatically.

Create Webhook

All fields marked with * are required

Name *



URL *



Server endpoint that will receive the webhook payload, for example:
"http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example:
"https://myLogin:myPassword@my_server/foo"

Secret



If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

Create Cancel

- Configured required tools in Jenkins: JDK, SonarQube Scanner, NodeJS, OWASP Dependency Check and Docker.

 **Jenkins** / Manage Jenkins ▾ / Tools

JDK installations

[+ Add JDK](#)

⋮ JDK

Name

☒ Install automatically ?

⋮ Install from adoptium.net ?

Version ?

[+ Add Installer](#)

 **Jenkins** / Manage Jenkins ▾ / Tools

SonarQube Scanner installations

[+ Add SonarQube Scanner](#)

⋮ SonarQube Scanner

Name

☒ Install automatically ?

⋮ Install from Maven Central

Version

[+ Add Installer](#)



NodeJS installations

+ Add NodeJS

⋮ NodeJS

Name

node25

☒ Install automatically ?

⋮ Install from nodejs.org

Version

NodeJS 25.7.0



Dependency-Check installations

+ Add Dependency-Check

⋮ Dependency-Check

Name

DP-Check

☒ Install automatically ?

⋮ Install from github.com

Version

dependency-check 12.2.0

+ Add Installer



Docker installations

+ Add Docker

⋮ Docker

Name

docker

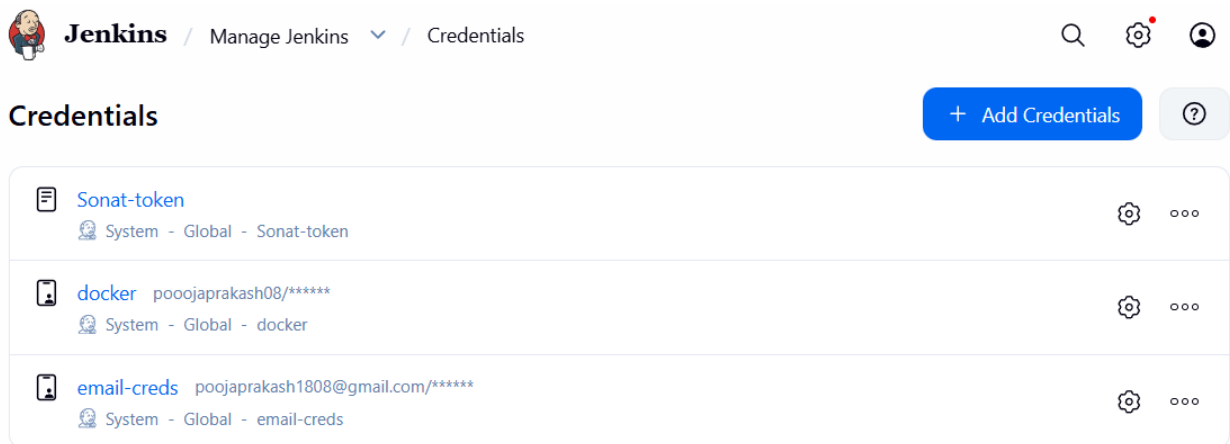
☒ Install automatically ?

⋮ Download from docker.com

Docker version ?

latest

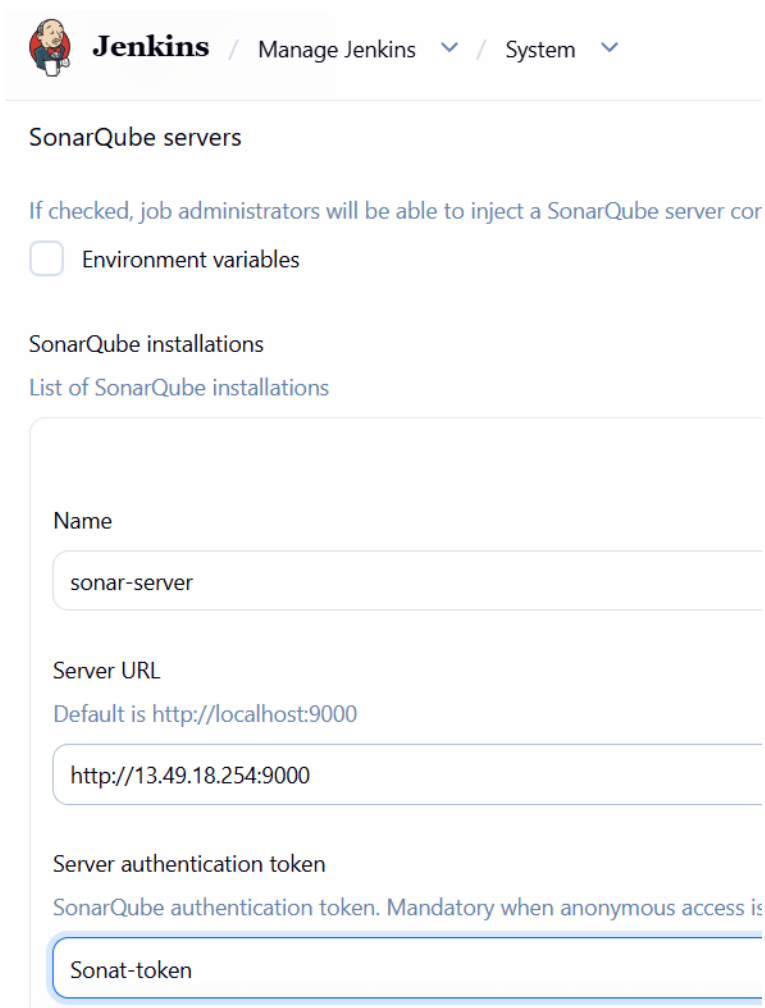
- Configured email credentials in Jenkins to send notifications for build success, failures, and updates.



The screenshot shows the Jenkins web interface. At the top, the breadcrumb navigation is 'Jenkins / Manage Jenkins / Credentials'. On the right, there are icons for search, settings, and a user profile. Below the breadcrumb, the title 'Credentials' is displayed on the left, and a blue button '+ Add Credentials' with a help icon is on the right. The main content area is a table listing three credentials:

Name	Description	Kind	Usage
Sonat-token	System - Global - Sonat-token	Text	System
docker	poojaprakash08/*****	Text	System
email-creds	poojaprakash1808@gmail.com/*****	Text	System

- Completed system configuration for SonarQube and email notifications in Jenkins.



The screenshot shows the Jenkins web interface for the 'System' configuration page. The breadcrumb navigation is 'Jenkins / Manage Jenkins / System'. The page title is 'SonarQube servers'. Below the title, there is a description: 'If checked, job administrators will be able to inject a SonarQube server configuration'. There is a checkbox labeled 'Environment variables' which is currently unchecked. Below this, the section 'SonarQube installations' is shown, followed by a link 'List of SonarQube installations'. The main configuration area is a form with the following fields:

- Name**: sonar-server
- Server URL**: Default is http://localhost:9000, the value entered is http://13.49.18.254:9000
- Server authentication token**: SonarQube authentication token. Mandatory when anonymous access is disabled, the value entered is Sonat-token

**Jenkins**[Manage Jenkins](#) / [System](#)

Extended E-mail Notification

SMTP server

smtp.gmail.com

SMTP Port

465

Advanced ^

Edited

Credentials

poojaprakash1808@gmail.com/***** (email-creds)

☒ Use SSL

☐ Use TLS

☒ Use OAuth 2.0

**Jenkins**[Manage Jenkins](#) / [System](#)

E-mail Notification

SMTP server

smtp.gmail.com

Default user e-mail suffix ?

Advanced ^

Edited

☒ Use SMTP Authentication ?

User Name

poojaprakash1808@gmail.com

For security when using authentication it is recommended to use OAuth 2.0

Password

.....

☒ Use SSL ?



SMTP Port ?

465

Reply-To Address

poojaprakash1808@gmail.com

Charset

UTF-8



Test configuration by sending test e-mail

Test e-mail recipient

poojaprakash1808@gmail.com

Email was successfully sent

Save

Apply



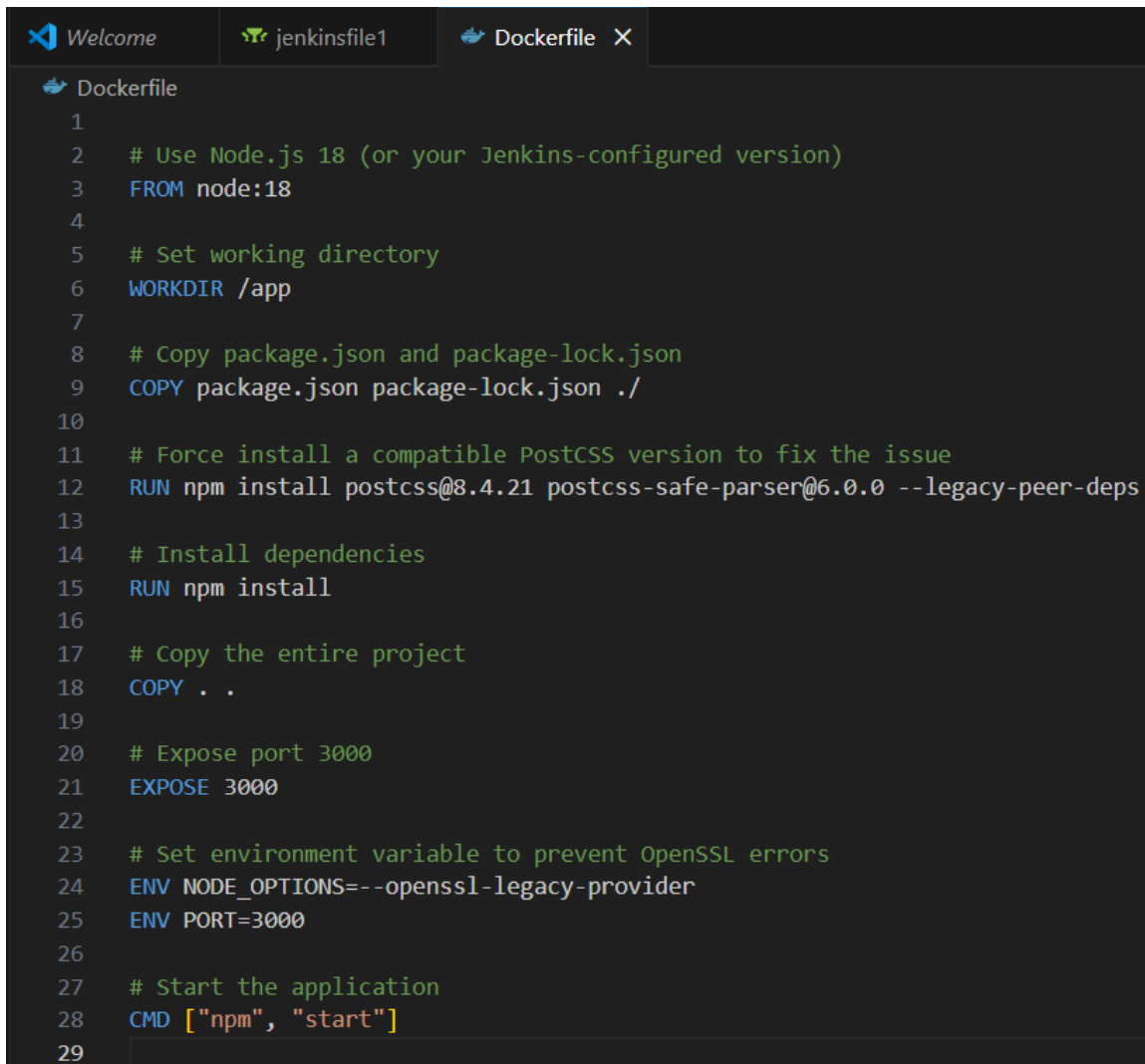
Default Triggers ^

Default Triggers ?

- ☐ Aborted
- ☒ Always
- ☐ Before Build
- ☐ Failure - 1st
- ☐ Failure - 2nd
- ☒ Failure - Any
- ☐ Failure - Still
- ☐ Failure - X
- ☐ Failure -> Unstable (Test Failures)
- ☐ Fixed
- ☐ Not Built
- ☐ Script - After Build
- ☐ Script - Before Build
- ☐ Status Changed
- ☒ Success
- ☐ Test Improvement

Phase 2: CI/CD Pipeline Execution

- Created a Dockerfile to containerize the application.



```
1
2  # Use Node.js 18 (or your Jenkins-configured version)
3  FROM node:18
4
5  # Set working directory
6  WORKDIR /app
7
8  # Copy package.json and package-lock.json
9  COPY package.json package-lock.json ./
10
11 # Force install a compatible PostCSS version to fix the issue
12 RUN npm install postcss@8.4.21 postcss-safe-parser@6.0.0 --legacy-peer-deps
13
14 # Install dependencies
15 RUN npm install
16
17 # Copy the entire project
18 COPY . .
19
20 # Expose port 3000
21 EXPOSE 3000
22
23 # Set environment variable to prevent OpenSSL errors
24 ENV NODE_OPTIONS=--openssl-legacy-provider
25 ENV PORT=3000
26
27 # Start the application
28 CMD ["npm", "start"]
29
```

- Created a Jenkinsfile that includes the following stages: Clone source code from GitHub, Perform SonarQube code analysis, install npm dependencies, Run OWASP dependency check, Perform Trivy file system scan, Build Docker image, Tag and push image to Docker Hub, Run Docker Scout image analysis, Deploy containerized application and Send email notifications.

```

Welcome jenkinsfile1 X Dockerfile
jenkinsfile1
1 pipeline {
2     agent any
3     tools {
4         jdk 'jdk17'
5         nodejs 'node25'
6     }
7     environment {
8         SCANNER_HOME = tool 'sonar-scanner'
9     }
10    stages {
11        stage('Clean Workspace') {
12            steps {
13                cleanWs()
14            }
15        }
16        stage('Checkout from Git') {
17            steps {
18                git branch: 'main', url: 'https://github.com/PoojaPrakash08/Book-My-Show.git'
19                sh 'ls -la' // Verify files after checkout
20            }
21        }
22        stage('SonarQube Analysis') {
23            steps {
24                withSonarQubeEnv('sonar-server') {
25                    sh '''
26                        $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=BMS \
27                        -Dsonar.projectKey=BMS
28                    '''
29                }
30            }
31        }
32        stage('Quality Gate') {
33            steps {
34                script {
35                    waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
36                }
37            }
38        }
39    }
40 }

```

```

38 }
39 stage('Install Dependencies') {
40     steps {
41         sh '''
42             cd bookmyshow-app
43             ls -la # Verify package.json exists
44             if [ -f package.json ]; then
45                 rm -rf node_modules package-lock.json # Remove old dependencies
46                 npm install # Install fresh dependencies
47             else
48                 echo "Error: package.json not found in bookmyshow-app!"
49                 exit 1
50             fi
51         '''
52     }
53 }
54 stage('OWASP FS Scan') {
55     steps {
56         dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit', odcInstallation: 'DP-Check'
57         dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
58     }
59 }
60 stage('Trivy FS Scan') {
61     steps {
62         sh 'trivy fs . > trivyfs.txt'
63     }
64 }

```

```

65     stage('Docker Build & Push') {
66         steps {
67             script {
68                 withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
69                     sh '''
70                     echo "Building Docker image..."
71                     docker build --no-cache -t poojaprakash08/bms:latest -f bookmyshow-app/Dockerfile bookmyshow-app
72
73                     echo "Pushing Docker image to registry..."
74                     docker push poojaprakash08/bms:latest
75                     '''
76                 }
77             }
78         }
79     }
80     stage('Deploy to Container') {
81         steps {
82             sh '''
83             echo "Stopping and removing old container..."
84             docker stop bms || true
85             docker rm bms || true
86
87             echo "Running new container on port 3000..."
88             docker run -d --restart=always --name bms -p 3000:3000 poojaprakash08/bms:latest
89
90             echo "Checking running containers..."
91             docker ps -a
92
93             echo "Fetching logs..."
94             sleep 5 # Give time for the app to start
95             docker logs bms
96             '''
97         }
98     }

```

```

99     }
100     post {
101         always {
102             emailx attachLog: true,
103                 subject: "'${currentBuild.result}'",
104                 body: "Project: ${env.JOB_NAME}<br/>" +
105                     "Build Number: ${env.BUILD_NUMBER}<br/>" +
106                     "URL: ${env.BUILD_URL}<br/>",
107                 to: 'poojaprakash1808@gmail.com',
108                 attachmentsPattern: 'trivyfs.txt,trivyimage.txt'
109         }
110     }
111 }

```

- Created a pipeline job named **BMS-Application** and configured it with the pipeline script.

Jenkins / All / New Item

Search Settings User

New Item

Enter an item name

BMS-Application

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Jenkins / BMS-Application / Configure

Search Settings User

Configure

 General

 Triggers

 Pipeline

 Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   tools {
4     jdk 'jdk17'
5     nodejs 'node25'
6   }
7   environment {
8     SCANNER_HOME = tool 'sonar-scanner'
9   }
10  stages {
11    stage('Clean Workspace') {
12      steps {
13        cleanWs()
14      }
15    }
16  }
```

try sample Pipeline...

☒ Use Groovy Sandbox ?

Save Apply

Jenkins

BMS-Application

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

SonarQube

Stages

Rename

Pipeline Syntax

BMS-Application

Add description

stage times:
~16min 49s)

Declarative: Tool Install	Clean Workspace	Checkout from Git	SonarQube Analysis	Quality Gate	Install Dependencies	OWASP FS Scan	Trivy FS Scan	Docker Build & Push	Deploy to Container	Declarative: Post Actions
191ms	370ms	1s	17s	403ms	2min 10s	9min 2s	5s	4min 59s	6s	345ms
191ms	370ms	1s	17s	403ms (paused for 3s)	2min 10s	9min 2s	5s	4min 59s	6s	345ms

SonarQube Quality Gate

BMS

Passed

server-side processing: Success

Latest Dependency-Check

- Successfully generated the SonarQube analysis report after pipeline execution.

sonarqube

Projects

Issues

Rules

Quality Profiles

Quality Gates

Administration

?

Search for projects...

A

My Favorites

All

Filters

Quality Gate

Passed

1

Failed

0

Reliability (0 Bugs)

A rating

0

B rating

0

C rating

0

D rating

1

E rating

0

Search by project name or key

Create Project

1 project(s)

Perspective: Overall Status

Sort by: Name

☆ BMS

Passed

Last analysis: 1 hour ago

Bugs

18

D

Vulnerabilities

0

A

Hotspots Reviewed

0.0%

E

Code Smells

116

A

Coverage

0.0%

Duplications

1.4%

Lines

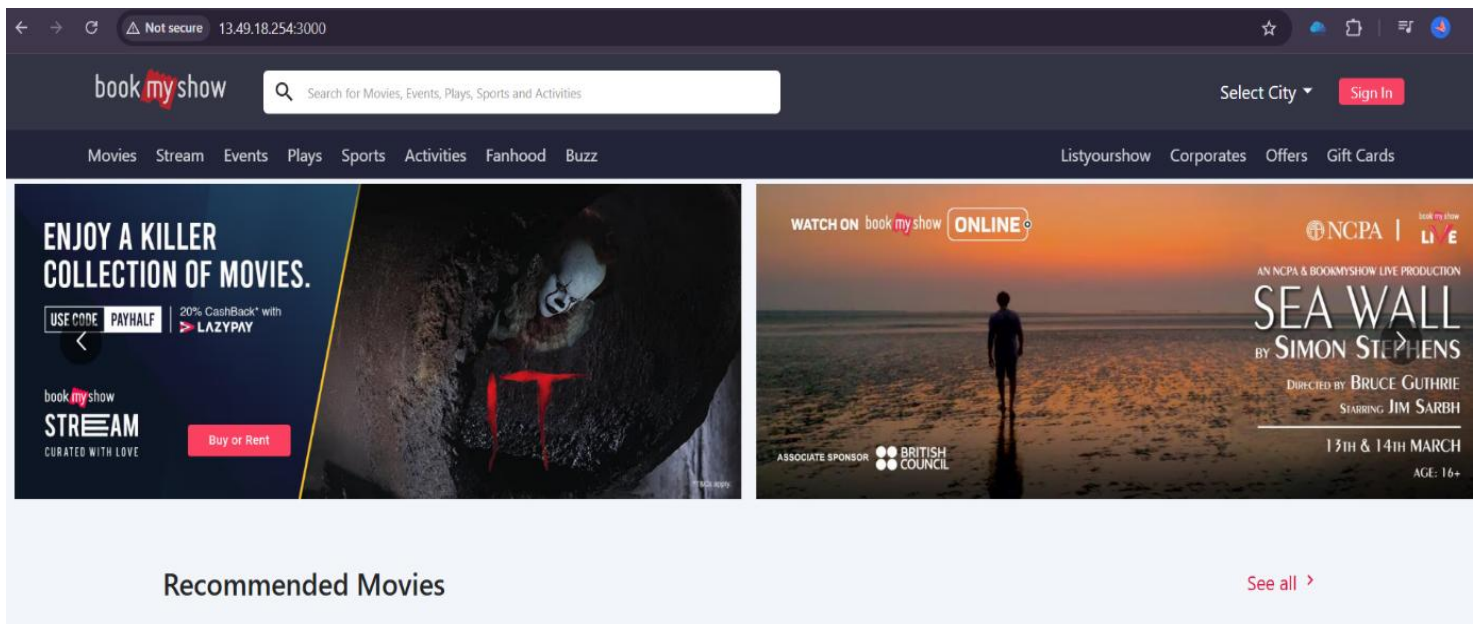
5.8k

S

JavaScript...

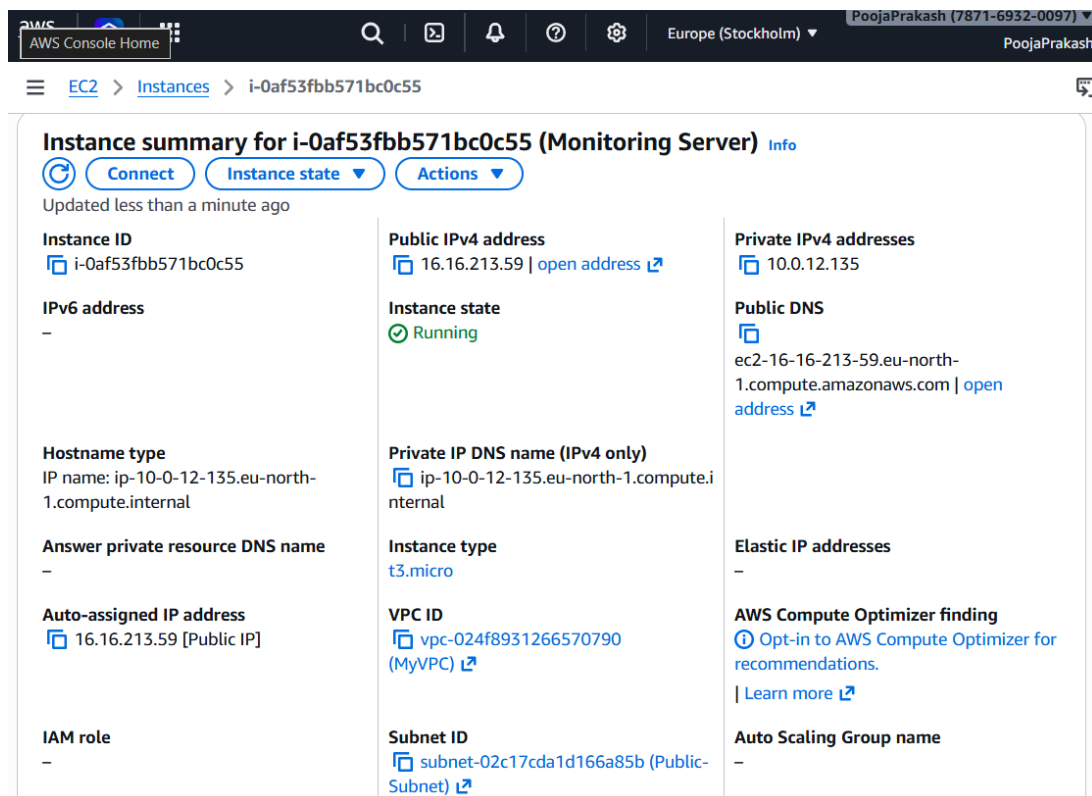
1 of 1 shown

- Accessed the Book My Show application using the public IP of the BMS-Server on port 3000.



Phase 3: Monitoring Setup

- Launched another Ubuntu EC2 instance named **Monitoring Server**.



- Installed Prometheus on the Monitoring Server.

```
ubuntu@ip-10-0-12-135:~$ prometheus --version
prometheus, version 2.47.1 (branch: HEAD, revision: c4d1a8beff37cc004f1dc4ab9d2e73193f51aaeb)
  build user:      root@4829330363be
  build date:      20231004-10:31:16
  go version:      go1.21.1
  platform:        linux/amd64
  tags:            netgo,builtinassets,stringlabels
ubuntu@ip-10-0-12-135:~$ sudo vi /etc/systemd/system/prometheus.service
ubuntu@ip-10-0-12-135:~$ sudo systemctl enable prometheus
Created symlink /etc/systemd/system/multi-user.target.wants/prometheus.service → /etc/systemd/system/prometheus.service.
ubuntu@ip-10-0-12-135:~$ sudo systemctl status prometheus
○ prometheus.service - Prometheus
   Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; vendor preset: enabled)
   Active: inactive (dead)
ubuntu@ip-10-0-12-135:~$ sudo systemctl start prometheus
ubuntu@ip-10-0-12-135:~$ sudo systemctl status prometheus
● prometheus.service - Prometheus
   Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-02-28 16:09:23 UTC; 20s ago
     Main PID: 1846 (prometheus)
       Tasks: 7 (limit: 1073)
      Memory: 17.5M
         CPU: 68ms
    CGroup: /system.slice/prometheus.service
            └─1846 /usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.p
```

- Installed Node Exporter to collect system-level metrics.

```
ubuntu@ip-10-0-12-135:~$ node_exporter --version
node_exporter, version 1.6.1 (branch: HEAD, revision: 4a1b77600c1873a8233f3ffb55afcedbb63b8d84)
  build user:      root@586879db11e5
  build date:      20230717-12:10:52
  go version:      go1.20.6
  platform:        linux/amd64
  tags:            netgo osusergo static_build
ubuntu@ip-10-0-12-135:~$ sudo vi /etc/systemd/system/node_exporter.service
ubuntu@ip-10-0-12-135:~$ sudo systemctl enable node_exporter
sudo systemctl start node_exporter
Created symlink /etc/systemd/system/multi-user.target.wants/node_exporter.service → /etc/systemd/system/node_exporter.service.
ubuntu@ip-10-0-12-135:~$ sudo systemctl status node_exporter
● node_exporter.service - Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-02-28 16:20:18 UTC; 14s ago
     Main PID: 1927 (node_exporter)
```


- Configured Prometheus to scrape metrics from Node Exporter and Jenkins.

```

[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target
StartLimitIntervalSec=500
StartLimitBurst=5
[Service]
User=prometheus
Group=prometheus
Type=simple
Restart=on-failure
RestartSec=5s
ExecStart=/usr/local/bin/prometheus \
--config.file=/etc/prometheus/prometheus.yml \
--storage.tsdb.path=/data \
--web.console.templates=/etc/prometheus/consoles \
--web.console.libraries=/etc/prometheus/console_libraries \
--web.listen-address=0.0.0.0:9090 \
--web.enable-lifecycle
[Install]
WantedBy=multi-user.target
~

```

- Updated and verified the syntax of the prometheus.yml configuration file.

```

# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1m.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1m.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['16.16.213.59:9100']

  - job_name: 'jenkins'
    metrics_path: '/prometheus'
    static_configs:
      - targets: ['13.49.18.254:8080']

```

```
ubuntu@ip-10-0-12-135:~$ cd /etc/prometheus/
ubuntu@ip-10-0-12-135:/etc/prometheus$ ls
console_libraries  consoles  prometheus.yml
ubuntu@ip-10-0-12-135:/etc/prometheus$ sudo vi prometheus.yml
ubuntu@ip-10-0-12-135:/etc/prometheus$ promtool check config /etc/prometheus/prometheus.yml
Checking /etc/prometheus/prometheus.yml
SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax
ubuntu@ip-10-0-12-135:/etc/prometheus$
```

- Accessed Prometheus via port 9090 and verified that all targets were in UP state.

← → ↺ ⚠ Not secure 16.16.213.59:9090/targets?search= ☆ 🌧 🗄 📄 🏠 ⋮

 Prometheus Alerts Graph Status ▾ Help   

Targets

All scrape pools ▾

All

Unhealthy

Collapse All

 Filter by endpoint or label

✓ Unknown

✓ Unhealthy

✓ Healthy

jenkins (1/1 up)

show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://13.49.18.254:8080/prometheus	UP	instance="13.49.18.254:8080" job="jenkins"	25.522s ago	8.380ms	

node_exporter (1/1 up)

show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://16.16.213.59:9100/metrics	UP	instance="16.16.213.59:9100" job="node_exporter"	21.938s ago	12.558ms	

prometheus (1/1 up)

show less

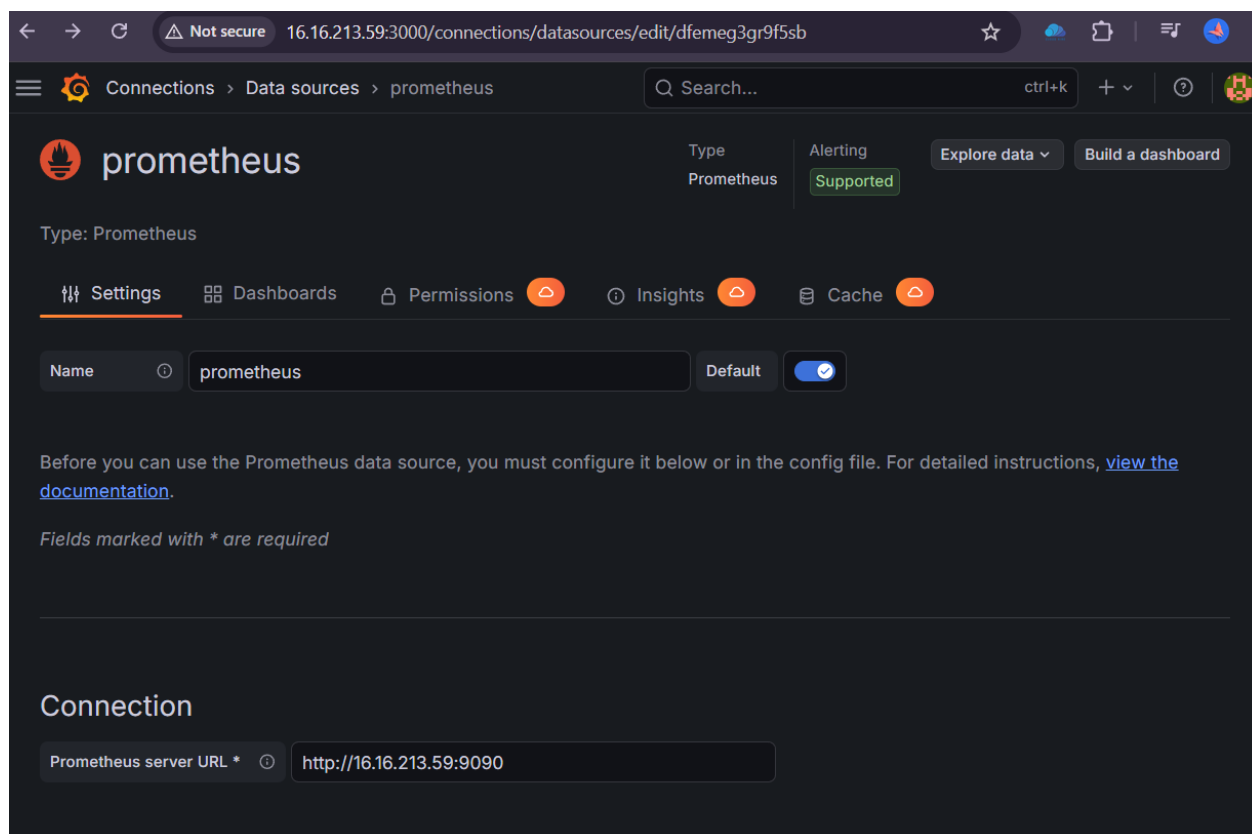
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	24.867s ago	3.726ms	

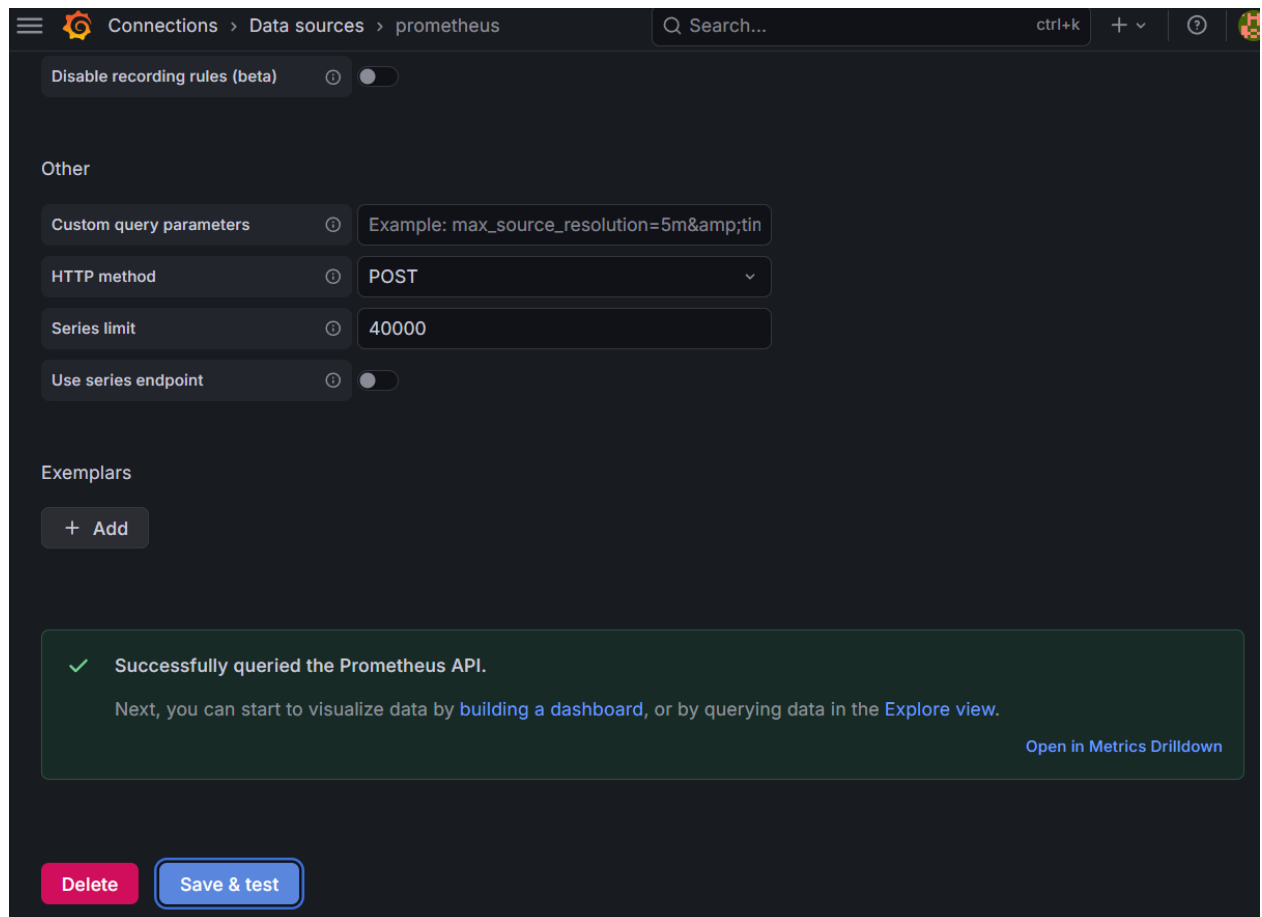
- Installed and started Grafana on the Monitoring Server.

```
ubuntu@ip-10-0-12-135:~$ sudo systemctl enable grafana-server
Synchronizing state of grafana-server.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable grafana-server
ubuntu@ip-10-0-12-135:~$ sudo systemctl start grafana-server
ubuntu@ip-10-0-12-135:~$ sudo systemctl status grafana-server
● grafana-server.service - Grafana instance
   Loaded: loaded (/lib/systemd/system/grafana-server.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-02-28 16:37:56 UTC; 21s ago
     Docs: http://docs.grafana.org
   Main PID: 3372 (grafana)
    Tasks: 15 (limit: 1073)
   Memory: 383.4M
      CPU: 5.692s
   CGroup: /system.slice/grafana-server.service
           └─3372 /usr/share/grafana/bin/grafana server --config=/etc/grafana/grafana.ini --pidfile=/run/

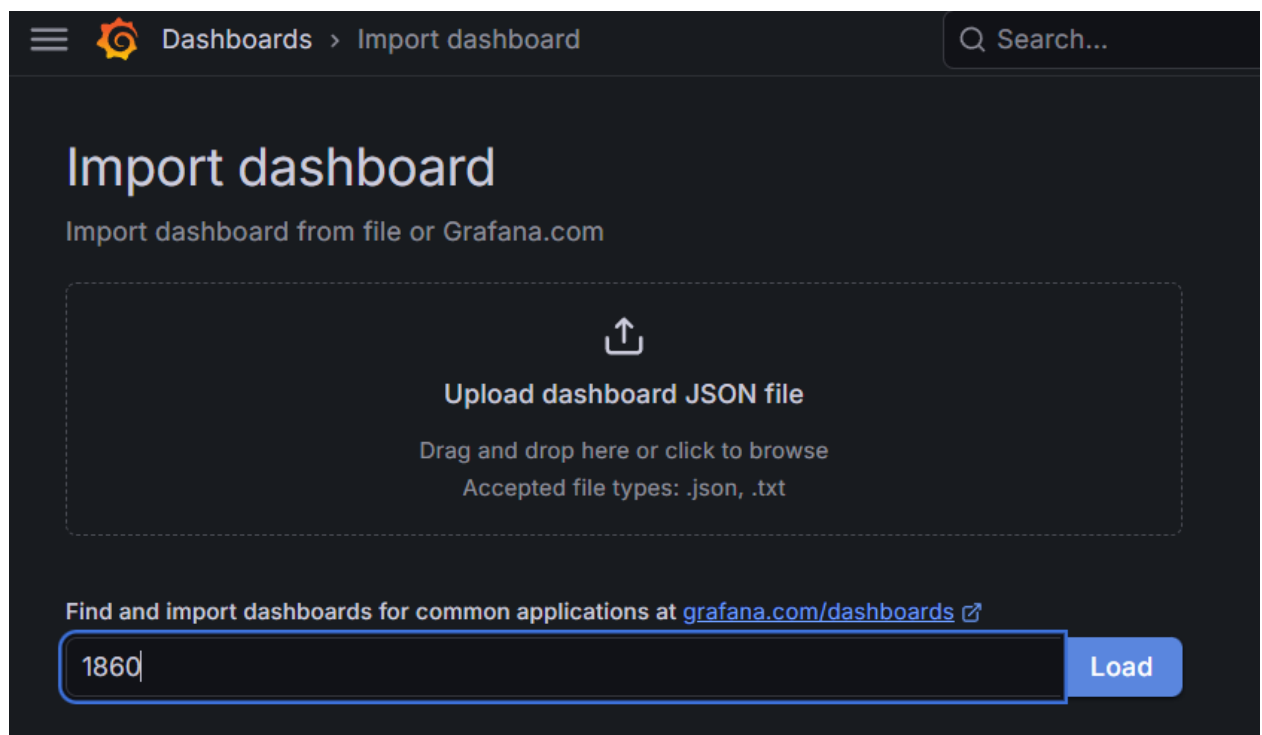
Feb 28 16:38:08 ip-10-0-12-135 grafana[3372]: logger=plugin.backgroundinstaller t=2026-02-28T16:38:08.52541Z level=info>
Feb 28 16:38:08 ip-10-0-12-135 grafana[3372]: logger=plugin.installer t=2026-02-28T16:38:08.860667561Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=installer.fs t=2026-02-28T16:38:09.037329927Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=plugins.registration t=2026-02-28T16:38:09.112872522Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=plugin.backgroundinstaller t=2026-02-28T16:38:09.11293Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=plugin.backgroundinstaller t=2026-02-28T16:38:09.11297Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=plugin.installer t=2026-02-28T16:38:09.69462856Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=installer.fs t=2026-02-28T16:38:09.773181593Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=plugins.registration t=2026-02-28T16:38:09.816211216Z level=info>
Feb 28 16:38:09 ip-10-0-12-135 grafana[3372]: logger=plugin.backgroundinstaller t=2026-02-28T16:38:09.81626Z level=info>
lines 1-21/21 (END)
ubuntu@ip-10-0-12-135:~$
```

- Accessed Grafana via port 3000 and added Prometheus as a data source.





- Imported the Node Exporter dashboard using Grafana Dashboard ID 1860.



Dashboards > Import dashboard

Q Search...

Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from [Grafana.com](#)

Published by

rfmoz

Updated on

2025-09-27 22:45:03

Options

Name

Node Exporter Full

Folder

Dashboards

Unique identifier (UID)

The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

rYdddlPWk

Change uid

Import

Cancel

Dashboards > Node Exporter Full

Q Search...

ctrl+k + ⓘ

☆ Edit Export ▾ Share ▾

Datasource prometheus ▾ Job node_exporter ▾ Nodename ip-10-0-12-135 ▾

« ⓘ Last 30 minutes ▾ » 🔍 ↺ Refresh 1m ▾

Instance 16.16.213.59:9100 ▾ [GitHub](#) [Grafana](#)

Quick CPU / Mem / Disk

Pres... ⓘ
CPU 0.3%
Mem 0.0%
I/O 0.1%

CPU... ⓘ
1.1%

Sys ... ⓘ
0.0%

RA... ⓘ
57.6%

SW... ⓘ
N/A

Root... ⓘ
11.8%

C...
2

R...
914 MiB

S...
0 B

R...
27 GiB

Uptime
1.0 hours

Basic CPU / Mem / Net / Disk

CPU Basic ⓘ
100%
80%
60%
40%
20%
0%
21:50 21:55 22:00 22:05 22:10 22:15
Busy System Busy User Busy Iowait Busy IRQs Busy Other Idle

Memory Basic ⓘ
896 MiB
768 MiB
640 MiB
512 MiB
384 MiB
256 MiB
128 MiB
0 B
21:50 21:55 22:00 22:05 22:10 22:15
Total Used Cache + Buffer Free Swap used

- Imported the Jenkins dashboard using Grafana Dashboard ID 9964.

Dashboards > Import dashboard

Import dashboard

Import dashboard from file or Grafana.com



Upload dashboard JSON file

Drag and drop here or click to browse

Accepted file types: .json, .txt

Find and import dashboards for common applications at grafana.com/dashboards

Dashboards > Import dashboard

Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from [Grafana.com](https://grafana.com)

Published by	haryan
Updated on	2023-08-24 15:04:53

Options

Name

Folder

 Dashboards

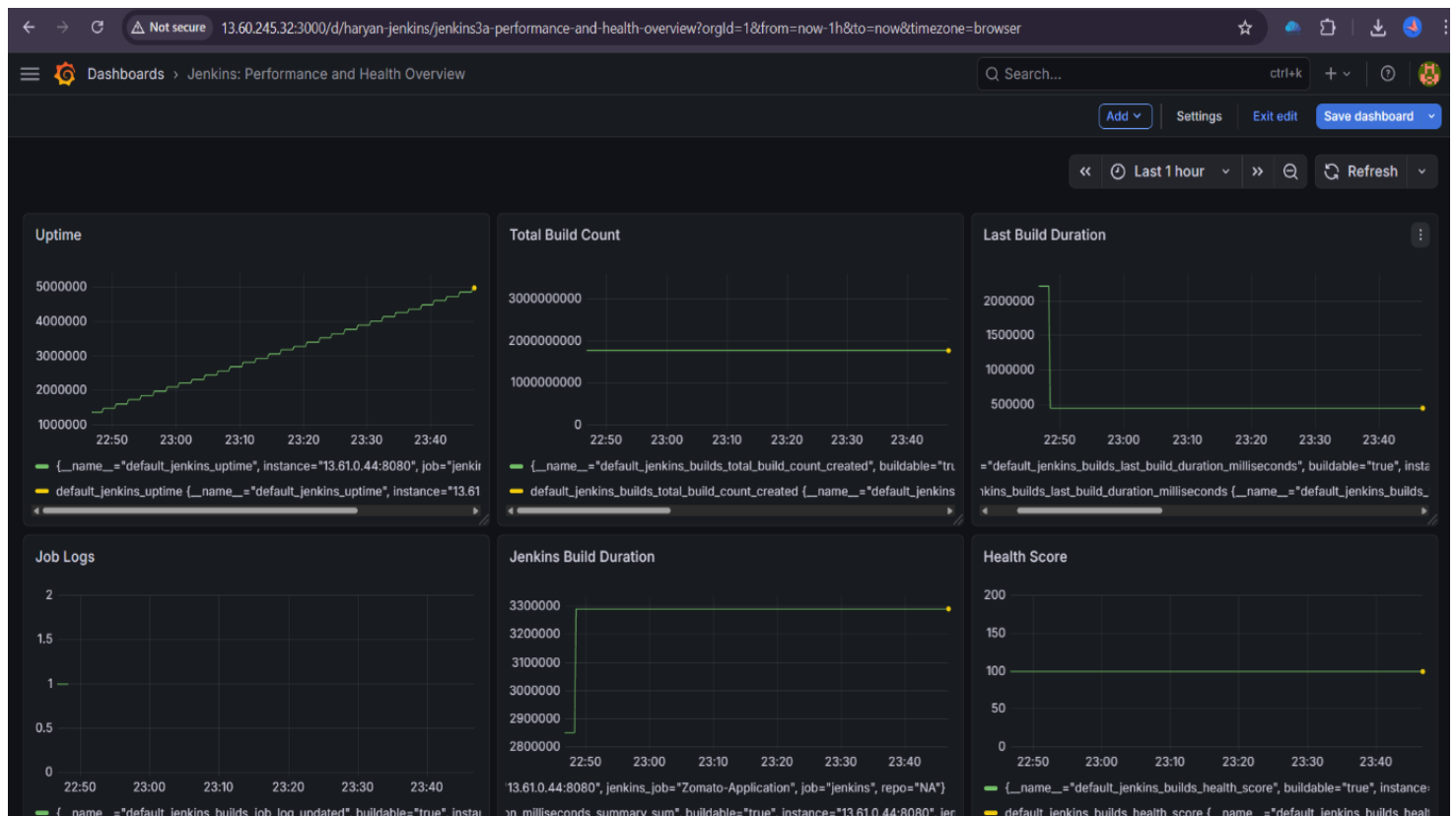
Unique identifier (UID)

The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

DS_PROMETHEUS

Select a prometheus data source

 prometheus



Phase 4: Kubernetes Deployment using Amazon EKS

- Created an IAM user named eks-user and attached necessary policies to manage the EKS cluster.

Permissions policies (6)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type

Search

All types

☐	Policy name	Type	Attached via
☐	AmazonEC2FullAccess	AWS managed	Directly
☐	AmazonEKS_CNI_Policy	AWS managed	Directly
☐	AmazonEKSClusterPolicy	AWS managed	Directly
☐	AmazonEKSWorkerPolicy	AWS managed	Directly
☐	AWSCloudFormationFullAccess	AWS managed	Directly
☐	IAMFullAccess	AWS managed	Directly

Step 1

Specify permissions

Step 2

Review and create

Specify permissions [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

Visual

JSON

Actions ▼



```
1  {  
2    "Version": "2012-10-17",  
3    "Statement": [  
4      {  
5        "Sid": "VisualEditor0",  
6        "Effect": "Allow",  
7        "Action": "eks:*",  
8        "Resource": "*"   
9      }  
10   ]  
11 }
```

**Edit statement****Select a statement**

Select an existing statement in the policy or add a new statement.

[+ Add new statement](#)

Step 1

Access key best practices & alternatives

Step 2 - optional

Set description tag

Step 3

Retrieve access keys**Retrieve access keys** [Info](#)**Access key**

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key

Secret access key



AKIA3ORXSTSQT4NS5MUC

***** [Show](#)**Access key best practices**

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download .csv file](#)[Done](#)

- Installed AWS CLI on the BMS-Server and configured it using the IAM user's access key and secret key.

```
You can now run: /usr/local/bin/aws --version
AWS Access Key ID [None]: AKIA30RXSTSQT4NS5MUC
AWS Secret Access Key [None]: esLoz2XTtwvCfGDpwWx4gylqhdhS7/Ywum7xYNfN
Default region name [None]: eu-north-1
Default output format [None]: json
root@ip-10-0-6-33:/home/ubuntu# aws --version
aws-cli/2.34.0 Python/3.13.11 Linux/6.14.0-1018-aws exe/x86_64.ubuntu.24
root@ip-10-0-6-33:/home/ubuntu#
```

- Installed kubectl and eksctl to create and manage the Kubernetes cluster.

```
root@ip-10-0-6-33:/home/ubuntu# kubectl version --client
Client Version: v1.35.2
Kustomize Version: v5.7.1
root@ip-10-0-6-33:/home/ubuntu# eksctl version
0.223.0
root@ip-10-0-6-33:/home/ubuntu#
```

- Created an EKS cluster named **pooja-eks** and verified its status in the AWS Console.

```
root@ip-10-0-6-33:/home/ubuntu# eksctl create cluster \
--name=pooja-eks \
--region=eu-north-1 \
--version=1.30 \
--vpc-public-subnets=subnet-02c17cda1d166a85b,subnet-061485a8046389ab0 \
--without-nodegroup
```

Amazon Elastic Kubernetes Service > Clusters

Cluster name ▲	Status ▼	Kubernetes version ▼	Support period ▼	Upgrad
pooja-eks	Active	1.30 Upgrade now	Extended support until July 23, 2026	Extend

- Associated an OIDC identity provider to enable IAM roles for service accounts.

```
root@ip-10-0-6-33:/home/ubuntu# eksctl utils associate-iam-oidc-provider \
> --region eu-north-1 \
> --cluster pooja-eks \
> --approve
2026-02-28 11:01:30 [i] will create IAM Open ID Connect provider for cluster "pooja-eks" in "eu-north-1"
2026-02-28 11:01:31 [✓] created IAM Open ID Connect provider for cluster "pooja-eks" in "eu-north-1"
root@ip-10-0-6-33:/home/ubuntu# npm --version
9.2.0
root@ip-10-0-6-33:/home/ubuntu#
```

- Created a managed Node Group in public subnets with required add-ons.

```
root@ip-10-0-6-33:/home/ubuntu# eksctl create nodegroup \
--cluster=pooja-eks \
--region=eu-north-1 \
--name=node2 \
--node-type=t3.small \
--nodes=3 \
--nodes-min=2 \
--nodes-max=4 \
--node-volume-size=20 \
--ssh-access \
--ssh-public-key=linux1 \
--managed \
--asg-access \
--external-dns-access \
--full-ecr-access \
--appmesh-access \
--alb-ingress-access \
--node-private-networking=false
```

- Verified worker nodes were successfully registered.

Amazon Elastic Kubernetes Service > Clusters > pooja-eks

Node groups (1) [Info](#) [Edit](#) [Delete](#) [Add](#)

Node groups implement basic compute scaling through EC2 Auto Scaling groups.

Filter node groups by property or value

Group name	Desired size	AMI release version	Launch template	Status
node2	3	1.30.14-20260224	eksctl-pooja-eks-nodegroup-node2 (1)	✓ Active

EC2 > Instances

Instances (4) Info Last updated 1 minute ago [Refresh](#) [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Find Instance by attribute or tag (case-sensitive) All states

Instance state = running Clear filters

☐	Name	Instance ID	Instance state	Instance type
☐	pooja-eks-node2-Node	i-0c52bc7e480cf522e	Running	t3.small
☐	BMS-Server	i-09cd586f4959846a3	Running	m7i-flex.large
☐	pooja-eks-node2-Node	i-0338c487f9e0e95cb	Running	t3.small
☐	pooja-eks-node2-Node	i-082aafb62e2998100	Running	t3.small

- Created Kubernetes YAML files for Deployment and Service.

```

! service.yml
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: bms-service
5    labels:
6      app: bms
7  spec:
8    type: LoadBalancer
9    ports:
10   - port: 80
11     targetPort: 3000
12   selector:
13     app: bms

```

```

! deployment.yml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: bms-app
5    labels:
6      app: bms
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: bms
12   template:
13     metadata:
14       labels:
15         app: bms
16     spec:
17       containers:
18       - name: bms-container
19         image: poojaprakash08/bms:latest
20         ports:
21         - containerPort: 3000

```

- Updated the Jenkinsfile to deploy the application into the Kubernetes cluster.

```

Welcome  jenkinsfile1  jenkinsfile2 X  ! deployment.yml  ! service.yml  Dockerfile
jenkinsfile2
1  pipeline {
2      agent any
3
4      tools {
5          jdk 'jdk17'
6          nodejs 'node25'
7      }
8
9      environment {
10         SCANNER_HOME = tool 'sonar-scanner'
11         DOCKER_IMAGE = 'poojaprakash08/bms:latest'
12         EKS_CLUSTER_NAME = 'pooja-eks'
13         AWS_REGION = 'eu-north-1'
14     }
15
16     stages {
17         stage('Clean Workspace') {
18             steps {
19                 cleanWs()
20             }
21         }
22
23         stage('Checkout from Git') {
24             steps {
25                 git branch: 'main', url: 'https://github.com/PoojaPrakash08/Book-My-Show.git'
26                 sh 'ls -la' // Verify files after checkout
27             }
28         }
29     }
30 }

```

```

97     stage('Deploy to EKS Cluster') {
98         steps {
99             script {
100                 sh '''
101                     echo "Verifying AWS credentials..."
102                     aws sts get-caller-identity
103
104                     echo "Configuring kubectl for EKS cluster..."
105                     aws eks update-kubeconfig --name $EKS_CLUSTER_NAME --region $AWS_REGION
106
107                     echo "Verifying kubeconfig..."
108                     kubectl config view
109
110                     echo "Deploying application to EKS..."
111                     kubectl apply -f deployment.yml
112                     kubectl apply -f service.yml
113
114                     echo "Verifying deployment..."
115                     kubectl get pods
116                     kubectl get svc
117                     ...
118                 '''
119             }
120         }
121     }

```

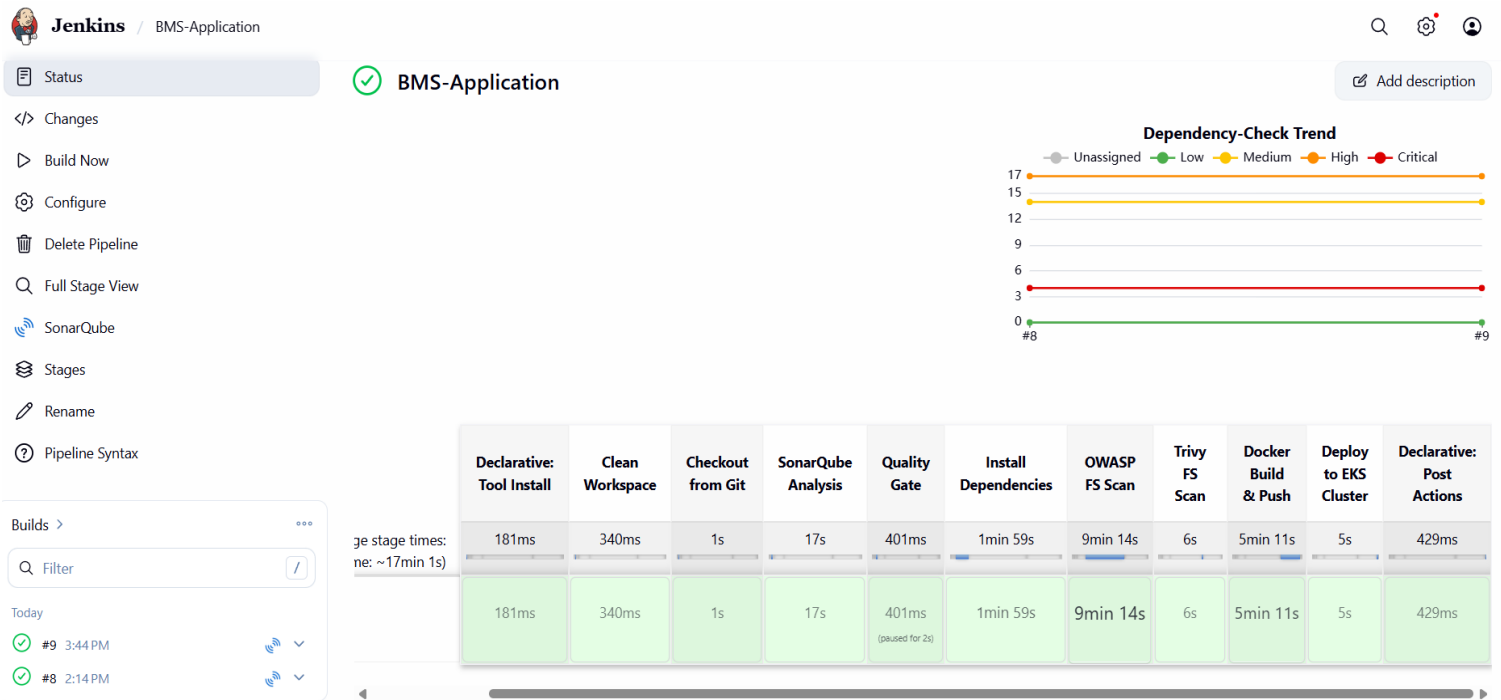
```
jenkins@ip-10-0-6-33:/home/ubuntu$ aws eks update-kubeconfig --name pooja-eks --region eu-north-1
Added new context arn:aws:eks:eu-north-1:787169320097:cluster/pooja-eks to /var/lib/jenkins/.kube/config
jenkins@ip-10-0-6-33:/home/ubuntu$
```

- Updated the pipeline job with the modified Jenkinsfile and rebuilt the pipeline.

The screenshot shows the Jenkins web interface for configuring a pipeline job named 'BMS-Application'. The left sidebar contains navigation links: 'General', 'Triggers', 'Pipeline' (which is highlighted), and 'Advanced'. The main area is titled 'Configure Pipeline' and includes a sub-header 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, a dropdown menu is set to 'Pipeline script'. The 'Script' section contains a Groovy script for a pipeline with the following content:

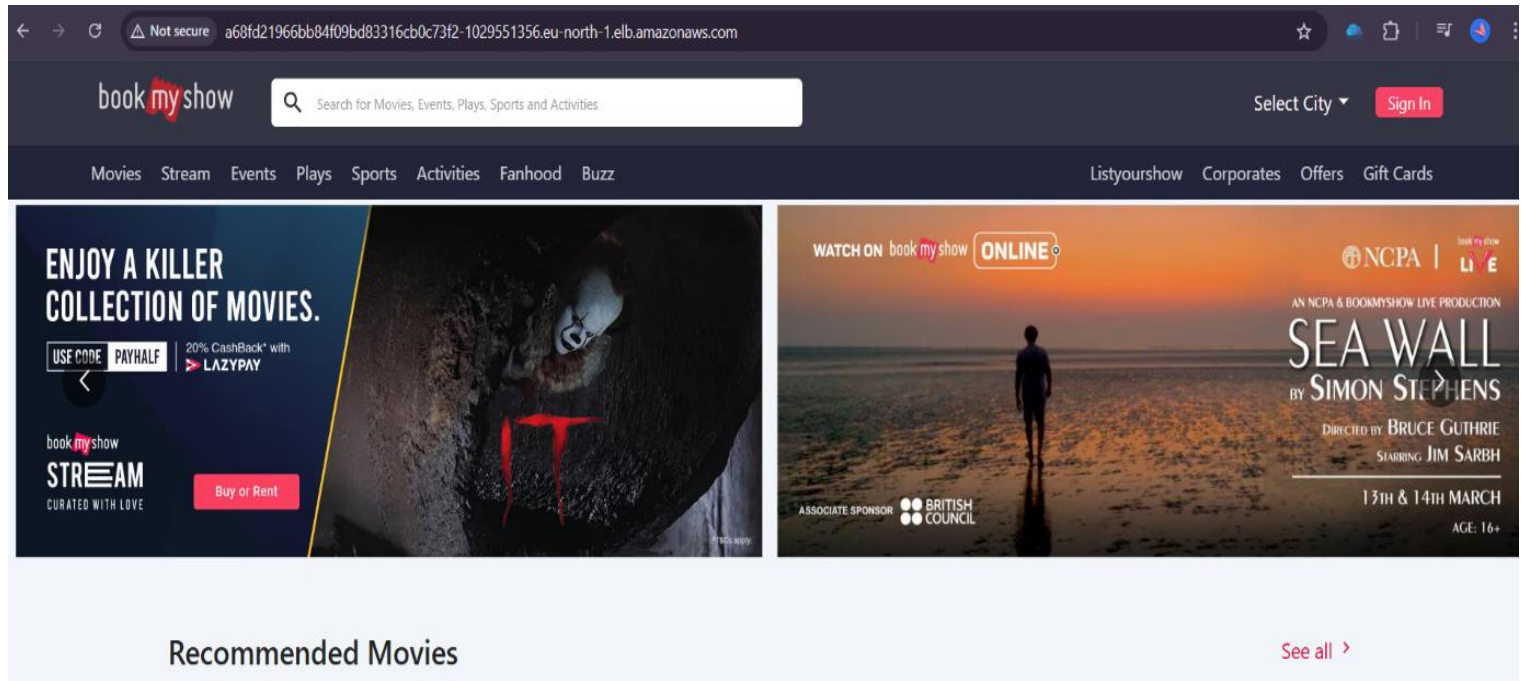
```
1 pipeline {
2     agent any
3
4     tools {
5         jdk 'jdk17'
6         nodejs 'node25'
7     }
8
9     environment {
10        SCANNER_HOME = tool 'sonar-scanner'
11        DOCKER_IMAGE = 'poojaprakash08/bms:latest'
12        EKS_CLUSTER_NAME = 'pooja-eks'
13        AWS_REGION = 'eu-north-1'
14    }
15 }
```

At the bottom of the configuration area, there is a checkbox labeled 'Use Groovy Sandbox' which is checked. Below the configuration area are two buttons: 'Save' and 'Apply'.



- Successfully accessed the Book My Show application using the Load Balancer URL provisioned by Kubernetes.

```
jenkins@ip-10-0-6-33:/home/ubuntu$ kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-10-0-14-90.eu-north-1.compute.internal Ready    <none>   5h49m   v1.30.14-eks-efcacff
ip-10-0-20-108.eu-north-1.compute.internal Ready    <none>   5h49m   v1.30.14-eks-efcacff
ip-10-0-8-156.eu-north-1.compute.internal Ready    <none>   5h49m   v1.30.14-eks-efcacff
jenkins@ip-10-0-6-33:/home/ubuntu$ kubectl get svc
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP
bms-service         LoadBalancer  172.20.202.110  a68fd21966bb84f09bd83316cb0c73f2-1029551356.eu-north-1.elb.ama
zonaws.com           LoadBalancer  172.20.202.110  a68fd21966bb84f09bd83316cb0c73f2-1029551356.eu-north-1.elb.ama
kubernetes           ClusterIP     172.20.0.1      <none>
```



- ✓ This project successfully demonstrates a complete DevOps lifecycle implementation for the Book My Show application, including CI/CD automation, containerization, security scanning, monitoring and Kubernetes-based deployment on AWS. The integration of Jenkins, Docker, SonarQube, OWASP, Trivy, Prometheus, Grafana and Amazon EKS ensures a scalable, secure and production-ready deployment architecture.